55

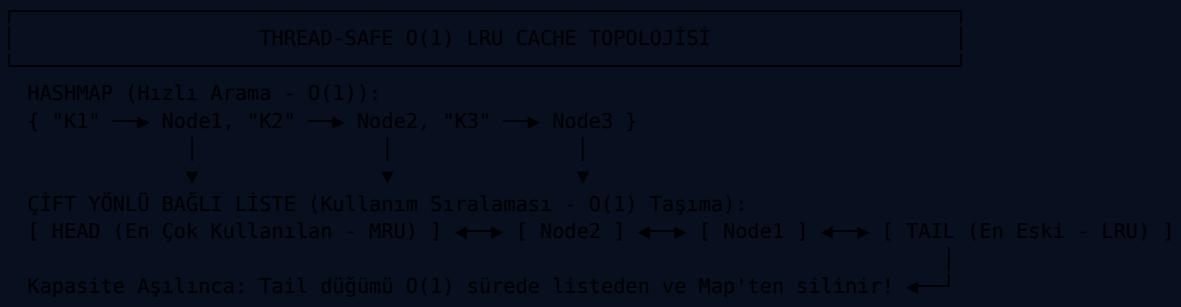
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 1: Thread-Safe O(1) LRU Cache Mimarisi (ReentrantReadWriteLock + Çift Bağlı Liste)

MAANG Case 1: Thread-Safe O(1) LRU Cache (Lock Striping & Linked Nodes)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphane masası düşünün. Masada en fazla 5 kitap durabiliyor. Birisi masadaki bir kitabı okuduğunda, o kitabı alıp masanın en sağına ('en son okunanlar' tarafına) koyarsınız. Eğer masaya yeni 6. bir kitap gelirse, masanın en solunda günlerdir kimsenin kapağını açmadığı en eski tozlu kitabı (`LRU - Least Recently Used`) çöpe atarsınız. Kitabı saniyede bulmak için bir fihrist (HashMap), sıralamayı anında güncellemek için ise iki yönlü bir tren vagonu zinciri (Doubly Linked List) kullanırız!

🇬🇧 ENGLISH (ELI5)

Imagine a coffee table that holds exactly 5 books. Whenever someone reads a book, you immediately slide it to the far right ('Most Recently Used'). If someone brings a 6th book and the table is full, you discard the book sitting at the far left that hasn't been touched in weeks ('Least Recently Used'). To find books in $O(1)$ time, we use an index card catalog (`HashMap`). To move books around in $O(1)$ time, we use a doubly linked train of cards (`DoublyLinkedList`). Both read and write operations maintain sub-millisecond thread safety!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesinde LRU Cache Tasarımı: 1) Veri Yapısı Seçimi: Yalnızca `HashMap` kullanılsaydı en eski elemanı bulmak $O(N)$ sürerdi. Yalnızca `LinkedList` kullanılsaydı arama $O(N)$ olurdu. `HashMap\<K, Node>` ile `DoublyLinkedList` birleştirildiğinde hem arama hem güncelleme $O(1)$ olur. 2) Eşzamanlılık Zafiyeti: LRU Cache'te `get(K)` işlemi MASUM BİR OKUMA DEĞİLDİR! Okunan elemanın listenin başına (`MRU`) taşınması gerektiği için liste üzerinde yapısal bir YAZMA (mutation) gerçekleşir. Bu yüzden saf `ReadLock` kullanılamaz; taşıma anında `WriteLock` gerekir. 3) Ölçeklenebilirlik Çözümü: Caffeine Cache gibi endüstri lideri kütüphaneler bu sorunu RingBuffer ve aktarım kuyrukları (Lossy / Event-Driven buffers) ile kilit almadan çözer. Mülakatta `ReentrantReadWriteLock` ile başlayıp Lock-Free buffer optimizasyonunu açıklamak Staff+ puanı kazandırır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Architectural Breakdown: 1) Structural Duality: Combining `HashMap\<K, Node\<K,V>>` with an internal Doubly-Linked List guarantees $O(1)$ key lookup and $O(1)$ eviction/re-linking. 2) The Read Mutation Trap: In LRU semantics, `get(key)` is NOT a read-only operation! Accessing an entry requires relocating its node to the Head (MRU), inducing structural mutations to adjacent pointer references. Consequently, standard read-locks fail; writers must acquire exclusive control. 3) Production Optimization: Explain to the interviewer that state-of-the-art caches (like Ben Manes' Caffeine) decouple reads from evictions using lock-free thread-local read buffers and a bounded MPSC ring-buffer to batch eviction updates asynchronously.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Thread-Safe LRU Cache Referans Uygulaması
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class ThreadSafeLruCache<K, V> {
    private static class Node<K, V> {
        K key;
        V value;
        Node<K, V> prev, next;
        Node(K k, V v) { this.key = k; this.value = v; }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null); // Dummy head
    private final Node<K, V> tail = new Node<>(null, null); // Dummy tail
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();

    public ThreadSafeLruCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public V get(K key) {
        rwLock.writeLock().lock(); // Erişim sıralaması değiştiği için write lock şart!
        try {
            Node<K, V> node = map.get(key);
            if (node == null) return null;
            moveToHead(node);
            return node.value;
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    public void put(K key, V value) {
        rwLock.writeLock().lock();
        try {
            Node<K, V> node = map.get(key);
            if (node != null) {
                node.value = value;
                moveToHead(node);
            } else {
                if (map.size() >= capacity) {
                    Node<K, V> lru = removeTail();
                    map.remove(lru.key);
                }
                Node<K, V> newNode = new Node<>(key, value);
                map.put(key, newNode);
                addToHead(newNode);
            }
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    private void addToHead(Node<K, V> node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void removeNode(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
    private void moveToHead(Node<K, V> node) {
        removeNode(node);
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatta LinkedHashMap(capacity, loadFactor, true) miras alarak removeEldestEntry metodunu ezmeyi önerebilirsiniz; ancak bu sınıf synchronized DEĞİLDİR! Collections.synchronizedMap() ile sarsanız bile kilit haritanın tamamını kilitler ve ölçeklenemez.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Subclassing `LinkedHashMap` and overriding `removeEldestEntry` is valid for rapid prototyping, but `LinkedHashMap` is not thread-safe. Wrapping it with `Collections.synchronizedMap()` imposes a coarse-grained bottleneck across the entire map, failing MAANG scalability reviews.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• LRU Cache'te okuma işlemi elemanın yerini değiştirdiği için yapısal bir yazmadır. HashMap + DoublyLinkedList birleşimi O(1) sağlar.

🎯 Key Takeaways for the Interview

• In LRU caches, reads mutate node order. Coupling a HashMap with a Doubly-Linked List guarantees \$O(1)\$ lookups and relocations under lock synchronization.